

ИИ ДЛЯ РАЗРАБОТЧИКОВ · СФУ

6 ак. часов

Итоговый проект: сервис бронирования

От контракта до контейнера за шесть академических часов,
методом Design First

| Максим Мигутин, тренер по ИИ · Академия Softline · 28 сентября – 7 октября 2026

Через обучение - к успеху!
Знания работают на вас

ЗАДАЧА

Что делаем

- Сервис записи на приём наподобие Calendly

Организатор заводит виды встреч и часы приёма, гость выбирает свободное время

- Гость не регистрируется

Открыл ссылку, увидел сетку слотов на неделю, нажал, ввёл имя и почту

- Занятый слот повторно не бронируется

Это главное правило, и оно защищено на двух рубежах

ЭТАЛОН

Готовый проект лежит в 05_Код/booking-reference. Подглядывать разрешено, копировать целиком нет.

На чём собираем

Слой	Технология
Контракт API	TypeSpec, из него собирается OpenAPI 3.1
Сервис	TypeScript в строгом режиме, Node.js 22, Fastify 5, проверка данных на Zod 4
Хранение	SQLite через better-sqlite3
Интерфейс	Vite, React 19, TypeScript, собственный CSS без библиотек компонентов
Тесты	Vitest для модульных, интеграционных и контрактных, Playwright для сквозных
Упаковка	Docker, docker compose, GitHub Actions

КАК УСТРОЕН РЕПОЗИТОРИЙ

Рабочие пространства npm: один package-lock.json в корне и четыре пакета внутри, contract, server, web и e2e. Одна команда npm install ставит зависимости всех четырёх.

Что должно быть в репозитории

Артефакт	Где лежит
Продуктовое решение	docs/pdr/0001-....md
Хотя бы одно архитектурное решение	docs/adr/0001-....md
Контракт API	contract/main.tsp и собранный contract/openapi.yaml
Код сервиса	server/src/, весь SQL в repository.ts
Интерфейс	web/src/, сборка Vite
Тесты, все зелёные	server/tests/, по возможности e2e/
Файл памяти агента	AGENTS.md
Запуск одной командой	Dockerfile и docker-compose.yml

Критерии оценки работы

Критерий	Что считается выполненным
Работает	<code>docker compose up -d</code> поднимает сервис, бронь создаётся
Соответствует контракту	Тест соответствия контракту зелёный
Защищено от двойной брони	Повторный запрос на тот же слот получает 409 и код <code>slot_taken</code>
Документировано	PDR и ADR написаны своими словами, не пересказ агента
Понятно	Можете объяснить любую строку своего кода

ОТДЕЛЬНО

Код, который вы не можете объяснить, к приёмке не принимается. Это не придирка, это и есть навык модуля.

Пять этапов на 6 ак. ч.

1 Спецификация

PDR и онтология. 45 минут

2 Контракт

TypeSpec и сборка оренари.yaml. 45 минут

3 Сервис

Схема базы, движок слотов, обработчики маршрутов. 90 минут

4 Интерфейс и тесты

Страница бронирования на React и зелёный набор тестов. 60 минут

5 Упаковка

Docker, непрерывная интеграция, README. 30 минут

1

Спецификация

Сорок пять минут, которые экономят два часа

ЭТАП 1

Что пишем до первой строки кода

- PDR

Задача с числами, роли, что решает сервис, сценарии, что не делаем

- Онтология

Четыре сущности: Activity, Schedule, Slot, Booking. Поля, типы, инварианты

- Одно правило, которое всегда истинно

На один слот не более одной действующей брони. Это инвариант И1

ПРИЁМ

Сначала три предложения своими словами, потом просим агента развернуть. Не наоборот.

Этап 1. Пишем спецификацию

- 1 Записать задачу в трёх предложениях без агента
- 2 Развернуть в PDR по пяти разделам, раздел «что не делаем» написать самостоятельно
- 3 Составить таблицу сущностей: поля, типы, обязательность, смысл
- 4 Записать список инвариантов и пронумеровать их. Минимум четыре

РЕЗУЛЬТАТ

Два файла в docs/ вашего проекта и понимание, из чего состоит сетка слотов. Шаблоны: 02_Студентам/04_Шаблоны/PDR-шаблон.md и Онтология-шаблон.md

Эталон: 01_Преподавателю/Решения_практикумов/modul6-marshrut/README.md

2

Контракт

Договор об API, из которого собирается всё остальное

ЭТАП 2

Минимальный контракт на TypeSpec

```
@service({ title: "Сервис бронирования тайм-слотов" })
namespace BookingService;

model Activity {
    id: int32;
    @minLength(1) @maxLength(100) name: string;
    @minValue(5) @maxValue(480) duration_minutes: int32;
}

@route("/api/activities")
interface Activities {
    @get op list(): Activity[];
    @post op create(@body body: ActivityCreate):
        { @statusCode statusCode: 201; @body body: Activity } | ValidationError;
}
```

Четыре модели и четыре интерфейса. Всё остальное появляется по мере надобности. Образец: 05_Код/booking-reference/contract/main.tsp

Собираем OpenAPI

```
npm install
npm run contract:build

> tsp compile .
Compilation completed successfully.

# появился contract/openapi.yaml, руками его не правим
```

Команда `npm run contract:build` нужна только после правки `contract/main.tsp`. Собранный файл коммитится вместе с кодом.

Этап 2. Собираем контракт

- 1 Описать в `main.ts` четыре модели: `Activity`, `Schedule`, `Slot`, `Booking`
- 2 Описать эндпоинты: список и создание для активностей, расписаний и броней, плюс список слотов
- 3 Собрать `openapi.yaml` командой `npm run contract:build`
- 4 Открыть собранный файл и убедиться, что все пути на месте

ЗАПАСНОЙ ВАРИАНТ

Компилятор TypeScript не встал: возьмите готовый `openapi.yaml` из `05_Код/booking-reference/contract/` и работайте от него.

Эталон: `01_Преподавателю/Решения_практикумов/module-marshrut/README.md`

3

Сервис

Полтора часа: схема базы, движок слотов, обработчики маршрутов

Куда что кладём

- | | | |
|---|---------------------------------|--|
| 1 | server/src/db.ts | Подключение к SQLite, схема таблиц и индексы |
| 2 | server/src/schemas.ts | Схемы Zod. Повторяют модели из контракта, типы выводятся через z.infer |
| 3 | server/src/slotEngine.ts | Расчёт слотов. Чистые функции, в базу не ходят |
| 4 | server/src/routes/ | По файлу на сущность. SQL здесь не пишем, он весь в repository.ts |
| 5 | server/src/app.ts | Сборка приложения Fastify и единая обработка ошибок |

Слоты считаются, а не хранятся

```
export function buildSlots(
  activity: Activity, schedules: Schedule[], busy: BusySlot[], dateFrom: string, dateTo: string,
): Slot[] {
  const busyKeys = new Set(busy.map((b) => `${b.date} ${b.start_time}`));
  const slots: Slot[] = [];
  for (const day of eachDay(dateFrom, dateTo)) {
    for (const schedule of schedules) {
      if (!schedule.weekdays.includes(isoWeekday(day))) continue;
      for (const t of timesForSchedule(schedule, activity.duration_minutes)) {
        slots.push({ activity_id: activity.id, schedule_id: schedule.id, date: day,
          start_time: t.start_time, end_time: t.end_time, is_free: !busyKeys.has(`${day} ${t.start_time}`) });
      }
    }
  }
  return slots.sort(compareSlots);
}
```

Файл server/src/slotEngine.ts, комментарии сокращены. Функция не знает про базу, поэтому её легко проверить тестом.

Защита от двойного бронирования

```
// Первый рубеж: проверка в коде
if (repository.findActiveBooking(activity.id, input.date, input.start_time) !== null) {
  throw errors.slotTaken();
}

// Второй рубеж: уникальный индекс в базе
try {
  reply.code(201);
  return repository.createBooking({ ... });
} catch (error) {
  if (isUniqueViolation(error)) {
    throw errors.slotTaken();
  }
  throw error;
}
```

Файл `server/src/routes/bookings.ts`. Один рубеж не спасает: два запроса могут пройти проверку одновременно.

Этап 3. Пишем сервис

- 1 Схема базы и схемы Zod по контракту. Прогнать тест соответствия контракту
- 2 Расчёт слотов отдельным файлом с чистыми функциями и тестами к ним
- 3 Обработчики: активности, расписания, слоты, брони
- 4 Защита от двойной брони на двух рубежах и проверка вручную: два одинаковых curl подряд

РЕЗУЛЬТАТ

Работает `npm run dev:server`, первый curl отвечает 201, второй такой же отвечает 409 с кодом `slot_taken`.
Образец: `05_Код/booking-reference/server/src/`

Эталон: `01_Преподавателю/Решения_практикумов/module-marshrut/README.md`

4

Интерфейс и тесты

Страница на Vite и React и зелёный набор тестов

Vite, React и никаких библиотек компонентов

- Точка входа и состояние экрана

web/src/main.tsx и web/src/App.tsx: активность, неделя, выбранный слот

- Компоненты по одному на блок экрана

ActivityPicker, WeekBar, SlotGrid, BookingPanel

- Обращения к API собраны в одном файле

web/src/api.ts: типы повторяют модели контракта

- Оформление своим CSS, один файл

web/src/styles.css: свободный слот синий, занятый серый

ОГРАНИЧЕНИЕ

Tailwind и готовые наборы компонентов не используем: интерфейс должен читаться целиком.

Что обязательно должно быть проверено

Что проверяем	Ожидаемый результат
Слоты по расписанию	Шаг 30 минут при длительности 30 минут даёт два слота в час
Слот не вылезает за рабочее окно	Длительность 45 минут при окне до 12:00 не даёт слот в 11:30
Бронь занимает слот	После брони слот приходит с is_free false
Двойная бронь	Второй запрос получает 409 и код slot_taken
Бронь на время вне сетки	422 и код slot_not_found
Отмена	Слот снова свободен
Соответствие контракту	Набор маршрутов и тела ответов совпадают с orepari.yaml

Этап 4. Интерфейс и тесты

- 1 Запустить `npm run dev:web` и сделать страницу: список активностей, сетка на неделю, форма брони
- 2 Проверить в браузере: занятый слот серый и не нажимается
- 3 Написать тесты по таблице выше, минимум шесть штук
- 4 Довести набор до зелёного: `npm test`, проверить типы: `npm run typecheck`

РЕЗУЛЬТАТ

Работающая страница бронирования на `http://localhost:5173` и зелёный прогон всех тестов. Образец:
`05_Код/booking-reference/web/src/`

Эталон: `01_Преподавателю/Решения_практикумов/modul6-marshrut/README.md`

5

Упаковка и публикация

Полчаса, после которых проект запускается у любого, и отметка, по которой он выходит версией

ЭТАП 5

Запуск одной командой

```
docker compose up -d --build

# http://localhost:8000

# что должно быть в docker-compose.yml:
#   сборка образа из Dockerfile
#   проброс порта 8000
#   именованный том для файла базы, чтобы брони не терялись
#   наполнение демонстрационными данными при старте
```

Проверьте на соседнем ноутбуке. Это единственная честная проверка упаковки. Образцы: 05_Код/booking-reference/Dockerfile и docker-compose.yml

Этап 5. Упаковываем

- 1 Собрать Dockerfile и docker-compose.yml. Можно попросить агента по образцу эталонного проекта
- 2 Добавить .github/workflows/ci.yml: npm ci, typecheck, npm test
- 3 Написать README: запуск на Windows, запуск на macOS, запуск в контейнере
- 4 Проверить README буквально: пройти по нему с нуля в чистой папке

РЕЗУЛЬТАТ

Проект, который запускается по вашему README без ваших подсказок. Образец упаковки: 05_Код/booking-reference

Эталон: 01_Преподавателю/Решения_практикумов/modul6-marshrut/README.md

Из образа в реестр одной отметкой

```
git tag v1.0.0
git push --tags

# дальше всё делает .github/workflows/release.yml:
#   npm test           проверки до публикации
#   changelog.ts       список изменений из истории коммитов
#   ghcr.io push       образ под тем же номером версии
#   gh-release         страница релиза со списком изменений
```

- Реестр образов у GitHub включён по умолчанию
Ни карты, ни отдельной учётной записи
- Тег образа совпадает с отметкой в истории
По адресу образа видно, какая версия где запущена

НАСТРОЙКА ИЗ МОДУЛЯ 4

Файл `release.yml` вы собирали в задании 9 модуля 4. Здесь он переносится без изменений.

Что он делает на этом этапе и чего не делает

✓ **Делает**

Собирает `Dockerfile` и `release.yml` по образцу, читает журнал упавшей сборки и называет причину, пишет черновик описания релиза

✗ **Не делает**

Не ставит отметку релиза, не сливает пул-реквест, не публикует образ. Эти три действия остаются за вами

НА ПРИЁМКЕ ЭТО СПРОСЯТ

«Что в вашем конвейере делает агент и где проходит граница». Ответ должен быть про конкретные три действия из вашего репозитория, а не про принципы.

Этап 9. Выпустить версию проекта

- 1 Перенести `release.yml` из модуля 4 в свой репозиторий
- 2 Поставить отметку `v1.0.0` и отправить её
- 3 Проверить: на вкладке Releases лежит версия, на вкладке Packages лежит образ
- 4 Дописать в `README.md` строку запуска опубликованного образа

ЧТО ЭТО ДАЁТ ПОРТФОЛИО

Ссылка, по которой ваш сервис запускается одной командой без сборки: `docker run -p 8000:8000 ghcr.io/логин/проект:v1.0.0`. На собеседовании это сильнее описания.

Живая ссылка вместо команды запуска

Площадка	Подходит ли нашему проекту
Render	Да. Запускает готовый образ из реестра: New → Web Service → Existing Image, адрес <code>ghcr.io/</code> <code>логин/проект:v1.0.0</code> , тариф Free, переменная <code>BOOKING_PORT = 10000</code>
Vercel	Нет. Там выполняются функции, а не долгоживущий процесс, и файл базы между запросами не сохраняется. Площадка выбирается под устройство приложения, а не по узнаваемости названия

ДВА ОГРАНИЧЕНИЯ, КОТОРЫЕ НАЗЫВАЕТЕ САМИ

Сервис засыпает после 15 минут простоя и просыпается около минуты. Постоянного диска на бесплатном тарифе нет: брони, созданные посетителями, после перезапуска пропадут. Демонстрационные данные наполняются при старте, поэтому пустым сервис не бывает.

Разбор: 01_Преподавателю/Решения_практикумов/module4-demo-sdlc/DEPLOY-PAAS.md

Как проходит приёмка, 8 минут у вашего монитора

- Отдельной защиты нет

Выступления перед группой и показа с проектора не будет

- Приёмка идёт в фоне

6 октября, во время самостоятельной работы: преподаватель садится рядом и открывает ваш репозиторий

- Разговор идёт по форме приёмки

Восемь критериев, запуск по README, `npm test`, два обязательных вопроса про конкретные строки вашего проекта и несколько вопросов по модулям курса

ЕСЛИ НЕ УСПЕЛИ

Добор приёмки в среду 7 октября: в 8:10 до занятия и во время практики модуля 7. Письменного теста на курсе нет: теория проверяется здесь же, на вашем проекте.

Что делать, если не успеваете

Ситуация	Запасной вариант
Не собрался контракт	Взять <code>орепарі.yaml</code> эталона и работать от него
Не хватает времени на интерфейс	Сдавать без страницы: сервис проверяется запросами <code>curl</code> и тестами
Кончились запросы к модели	Дописывать самостоятельно, объём небольшой
Не встал Docker	Сдавать запуск через <code>npm run build</code> и <code>npm start</code> на порте 8000
Не встал Playwright	Сдавать без сквозных тестов, интеграционные обязательны
Тесты красные и не чинятся	Сдать красный тест с описанием того, что расходится

ПРО ЧЕСТНОСТЬ

**Лучше мало кода, который вы понимаете,
чем много кода, который принёс агент.**

На приёмке спрашивают понимание. Это же будет и на первой работе.

Итог модуля 6

- Прошли полный путь Design First: спецификация, контракт, код, тесты, упаковка
- Собрали работающий сервис с защитой главного правила предметной области
- Умеем объяснить каждое принятое решение и каждую строку своего кода
- Проверили, что проект запускается по README у другого человека

Что дальше

Модуль 7. Работа с legacy-кодом, 4 ак. ч.

- Раздатка модуля 6

02_Студентам/02_Раздатки/Модуль_6_Итоговый_проект.pdf

- Эталонный проект

05_Код/booking-reference: README.md, AGENTS.md, docs/, contract/, server/tests/

- К следующему занятию

Скачать проект legacy-demo и убедиться, что `npm ci` и `npm run legacy` отрабатывают